

Lab. 2
Relational Data Modeling & Schema Design

Content

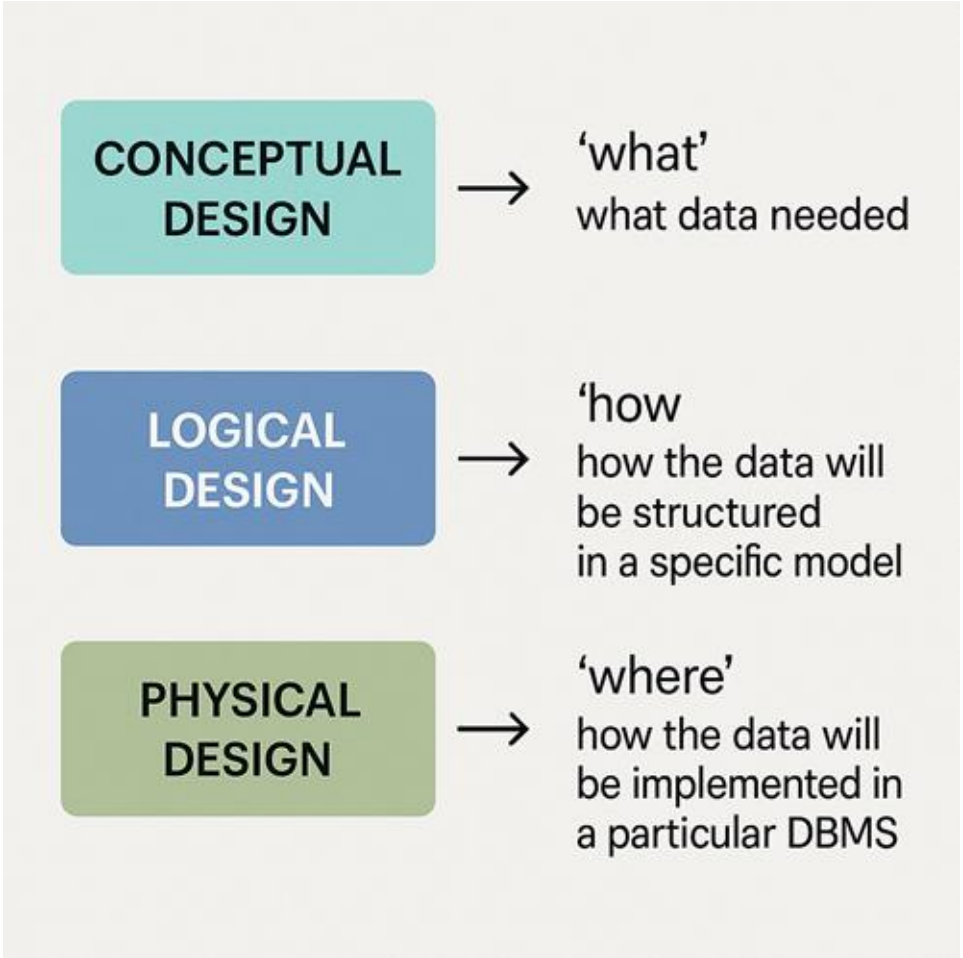
1. Recap: from temporary objects to a permanent schema
2. Conceptual, logical & physical design
3. Tables, rows, columns and data types
4. The ER model: entities, attributes, keys & relationships
5. Cardinality and ER diagram notation
6. Normalization
7. Conceptual Design to Database schema
8. Connecting to MySQL Workbench and a tour of the interface
9. Where We're Going Next
10. Summary

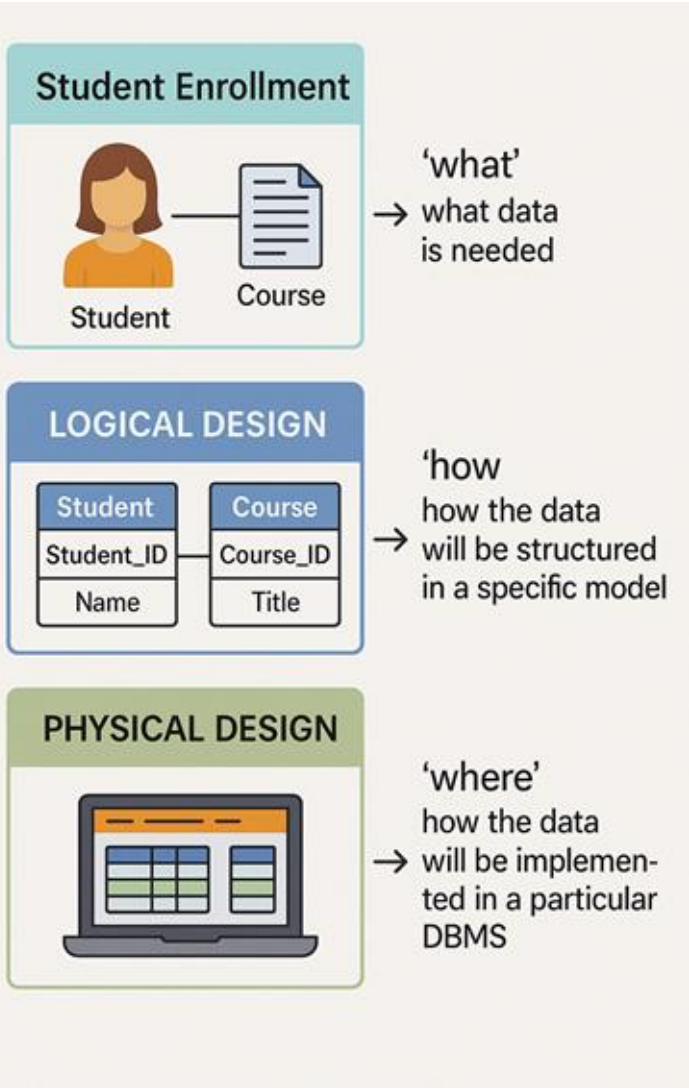
Recap: From Lab 1 to Lab 2

- ✓ In Lab 1 you stored records as objects inside an array — a temporary, in-memory database.
  - ❖ let students = [{ id: 1, name: 'Somchai', age: 20 }, ...];
- ✓ That data vanished when the program stopped. Now we design a permanent structure for it.
- ✓ A database schema is the blueprint: the tables, their columns, the type of each column, and how the tables relate.
- ✓ Today is about design — in Lab 3 we fill these tables with data and query them.

Conceptual, Logical & Physical Design

- Good design moves through three levels, from idea to implementation:
- We work top-down: settle the what, then the how, then the where.





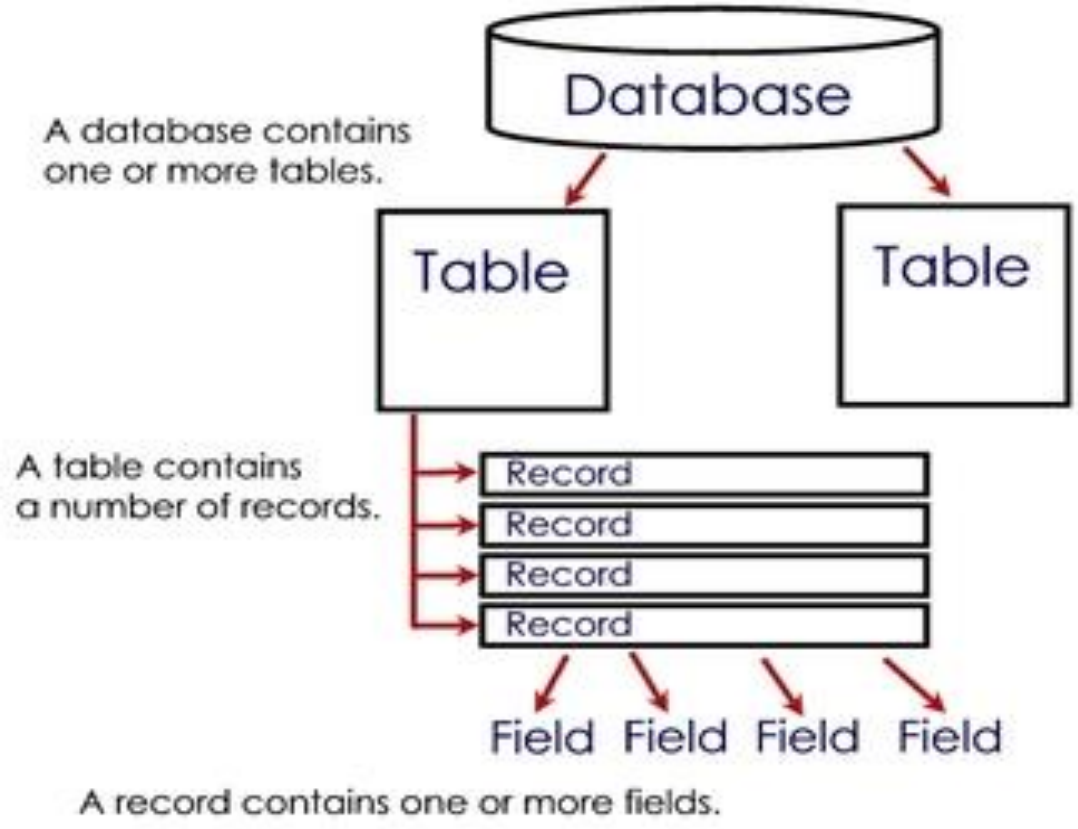
Tables, rows, columns and data types

What is Data Modeling?

- Data modeling means deciding how to organize data into tables before we store anything.
- We ask three questions:
  - ✓ What things (entities) do we store? e.g. students, courses.
  - ✓ What facts (attributes) describe each one? e.g. a student's name and age.
  - ✓ How are these things related? e.g. a student enrolls in many courses.
- Good design now prevents duplicated, inconsistent and hard-to-query data later.

Tables, rows, columns and data types (Contd.)

Anatomy of a Table (Collection of tables: Database)



ID	*name*	*dept_name*	*tot_cred*
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

Tables, rows, columns and data types (Contd.)

Choosing the Right Data Type: *Every column has a type that fixes what it can hold.*

- **Common MySQL types:**
  - INT — whole numbers (id, age)
  - VARCHAR(n) — short text, up to n chars
  - DATE / DATETIME — dates & times
  - DECIMAL(p,s) — exact money values
- Pick the smallest type that fits

INT	VARCHAR(50)
20	'Somchai'
DATE	**DECIMAL(8,2)**
2026-08-04	199.00

The ER model: entities, attributes, keys & relationships (Contd.)

The ER Model: Entities & Entity Sets

- The Entity-Relationship (ER) model describes data as entities and the relationships between them.
- **Entity: one real-world thing we store data about: a particular student, a particular book.**
- **Entity set: the collection of all entities of the same kind: all students, all books.**
  - An entity set becomes a table; one entity becomes a row.
- Each entity is described by its attributes, the columns of its table.
- We design entities and relationships first, then translate them into tables.

The ER model: entities, attributes, keys & relationships (Contd.)

Types of Attributes

- Attributes are the properties of an entity. They come in several kinds:
- **Simple — an atomic value, not divided further (age; a phone number).**
- **Composite — made of smaller parts (full_name = first_name + last_name).**
- **Derived — computed from other data, not stored (age from date_of_birth).**
- **Single-valued — holds exactly one value (a student's ID).**
- **Multi-valued — can hold several values (a person's phone numbers).**
  - ✓ In tables: split composites into columns; compute derived values in queries; give a multi-valued attribute its own related table.

The ER model: entities, attributes, keys & relationships (Contd.)

Lab Exercise 1: Model an entity & choose data types

- For a library, list the attributes a book needs (title, author, year, ISBN, copies).
- Beside each attribute, write the MySQL data type you would use and why.
- Mark which attribute should be the primary key.

Starter code:

```
book
  title   -> VARCHAR(?)
  author  -> VARCHAR(?)
  year    -> ____
  isbn    -> ____
  copies  -> ____
  PRIMARY KEY -> ____
```

The ER model: entities, attributes, keys & relationships

Primary Keys

- ✓ A key is an attribute (or set) that uniquely identifies an entity.
- ✓ A primary key is a column whose value uniquely identifies each row, no two rows share it.
- ✓ **Rules:**
  - ❑ Unique, and never empty (NOT NULL).
  - ❑ Usually a simple integer id.
  - ❑ AUTO_INCREMENT lets MySQL number new rows for you.
- ✓ Why it matters: it is how other tables refer to this row, the basis of relationships.

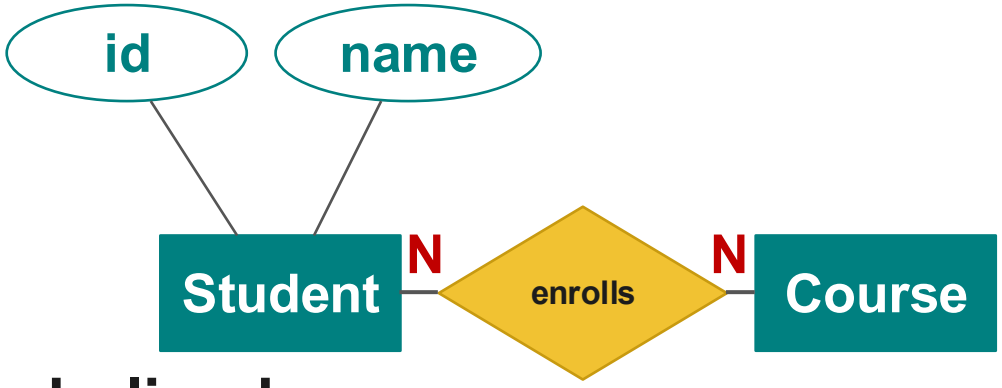
Cardinality and ER diagram notation

Relationships & Cardinality

- A relationship is an association between entities — a student enrolls in a course; an author writes a post.
- Cardinality says how many entities on one side relate to the other:
  - One-to-one (1:1) — one department has one head.
  - One-to-many (1:N) — one customer places many orders.
  - Many-to-one (N:1) — the same link seen from the other side.
  - Many-to-many (N:N) — a student takes many courses; a course has many students (needs a junction table).

Cardinality and ER diagram notation (Contd.)

ER Diagram Notation



Rectangle = entity
Ellipse = attribute
Diamond = relationship

- The classic ER notation uses three shapes:
- **Rectangle = an entity (a table).**
- **Ellipse = an attribute; the primary key is underlined.**
- **Diamond = a relationship between entities.**
- Cardinality (N, N) is written on the relationship lines.
  - This course mostly uses the compact box form (next slides) — both say the same thing (Both Chen and Crow's foot notations are used & accepted).

Cardinality and ER diagram notation (Contd.)

Relationships: One-to-Many

One student has many enrollments; each enrollment belongs to exactly one student.

Chen's Notation.



Crow's foot Notation.



A foreign key is a column holding the primary-key value of a row in another table, it links the two tables.

Cardinality and ER diagram notation (Contd.)

Lab Exercise 2: Add a relationship with a foreign key

- A blog has authors and posts. One author writes many posts.
- Design the two tables: list the columns of each.
- Show which column in posts is the foreign key, and which table's primary key it points to.

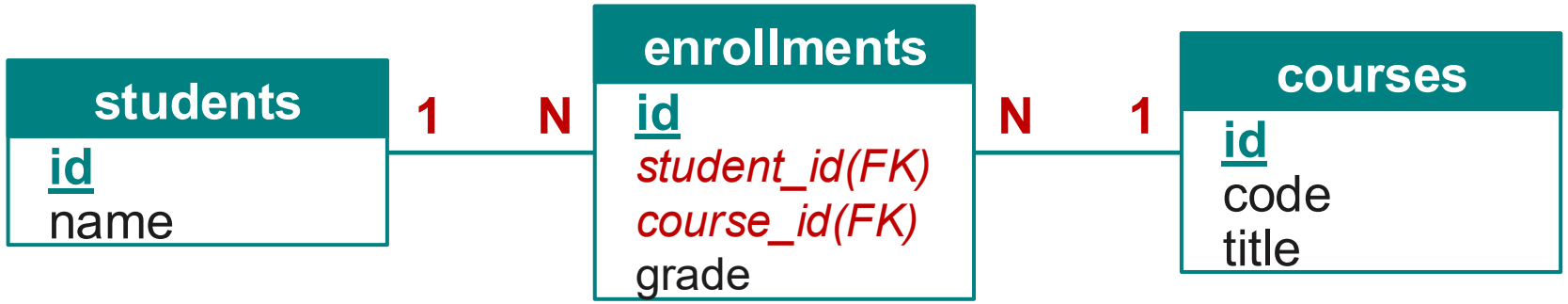
Starter code:

```
authors                posts
  id (PK)                id (PK)
  name                   title
  ...                    author_id (FK -> ____)
                         ...
```

Cardinality and ER diagram notation (Contd.)

Many-to-Many via a Junction Table

A student takes many courses, and a course has many students. We model this with a junction table.



The junction table (enrollments) holds two foreign keys, one to each side: turning many-to-many into two one-to-many links.

Cardinality and ER diagram notation (Contd.)

Reading an ER Diagram

- An Entity-Relationship (ER) diagram is a picture of your schema.
  - Each box is an entity (a table).
  - The lines inside list its attributes (columns).
  - The underlined field is the primary key; FK marks a foreign key.
  - Lines between boxes show relationships, labelled 1 or N for cardinality.
- Sketching an ER diagram before writing SQL helps you catch design problems early.

Normalization

Spotting Redundancy

student	course	instructor
Somchai	CSS326	Dr. Somchart
Suda	CSS326	Dr. Somchart
Anong	CSS227	Dr. Suparchoek

The instructor of CSS326 is repeated on every row. If it changes, we must fix many rows and might miss one. That is redundancy.

Normalization (Contd.)

The Fix

- Normalization splits data so each fact is stored once, in one place.
- **Three quick rules of thumb:**
  - 1NF: one value per cell, no repeating groups.
  - 2NF: every column depends on the whole primary key.
  - 3NF: no column depends on another non-key column.
- Our fix: a courses table stores each course and its instructor once; enrollments links students to courses.

Normalization (Contd.)

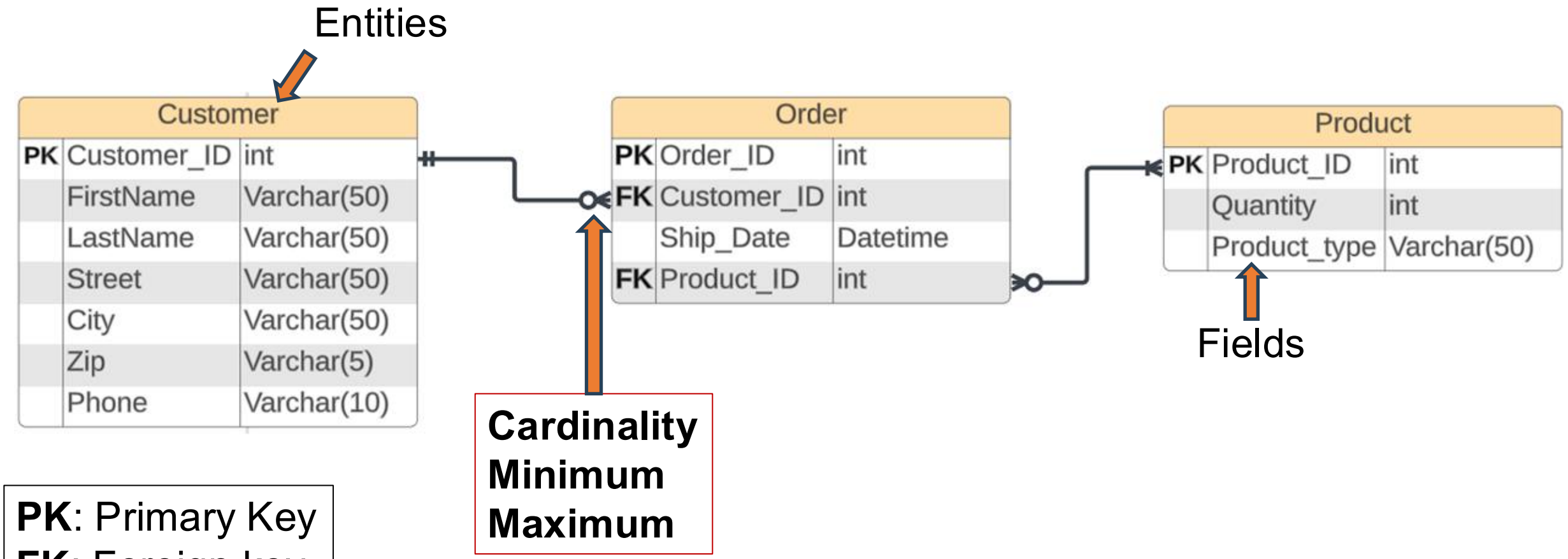
Lab Exercise 3: Normalize a redundant table

- Take the student / course / instructor table from the slide.
- Split it into separate tables so each instructor is stored only once.
- List the columns of each new table and mark the keys (PK and FK).

Starter code:

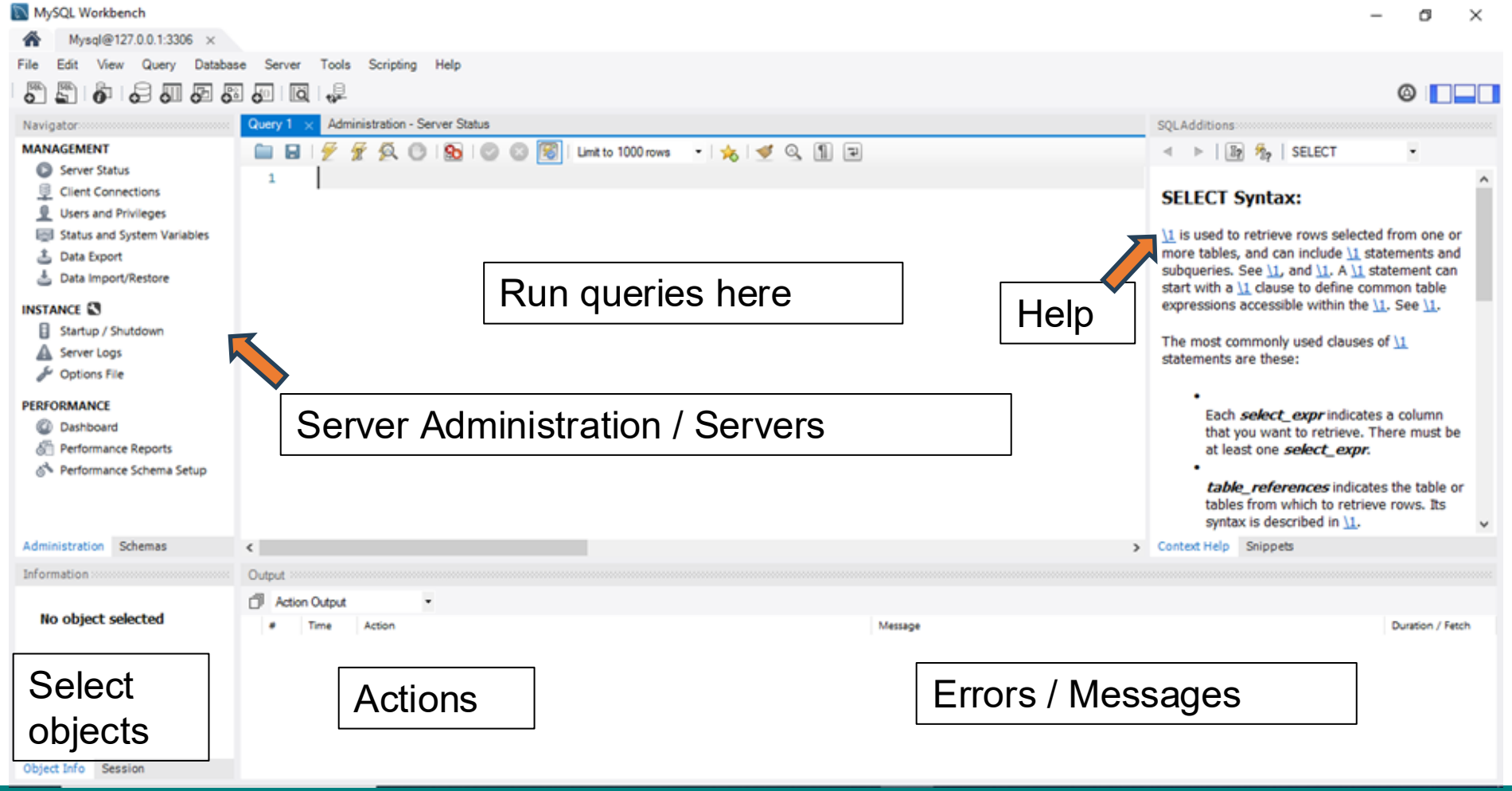
```
students        courses          enrollments
  id (PK)       id (PK)        student_id (FK)
  name           code           course_id (FK)
                instructor       ...
```

Conceptual Design to Database schema



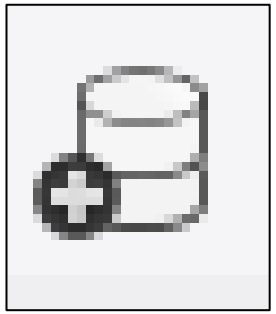
PK: Primary Key
FK: Foreign key

Connecting to MySQL Workbench and a tour of the interface

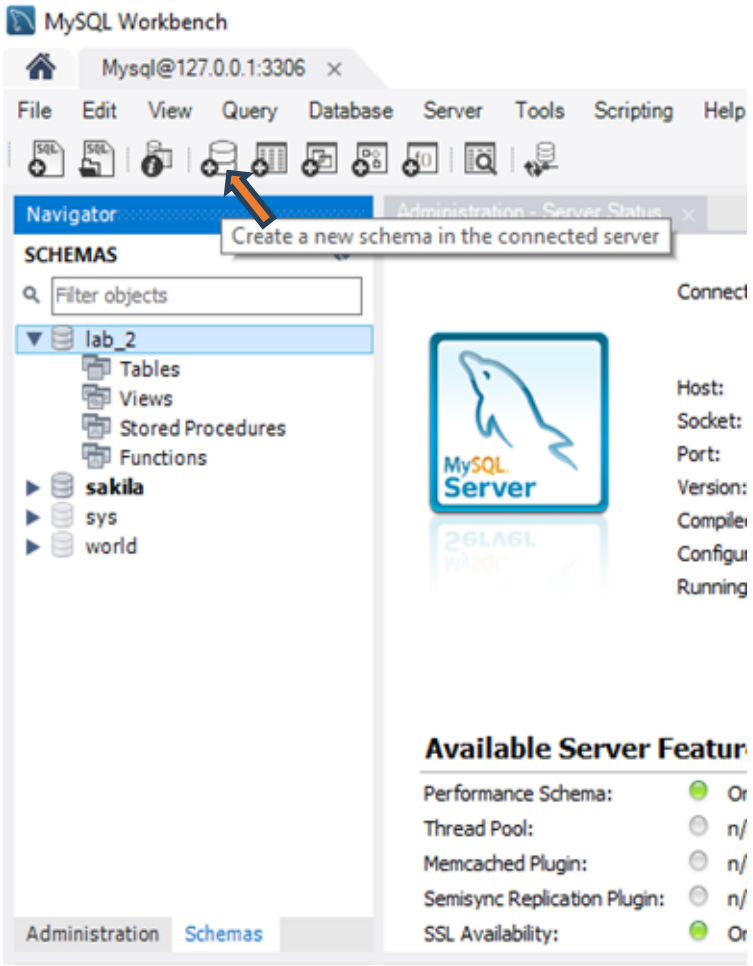


Connecting to MySQL Workbench and a tour of the interface (Contd.)

Create a new schema “Lab_2”

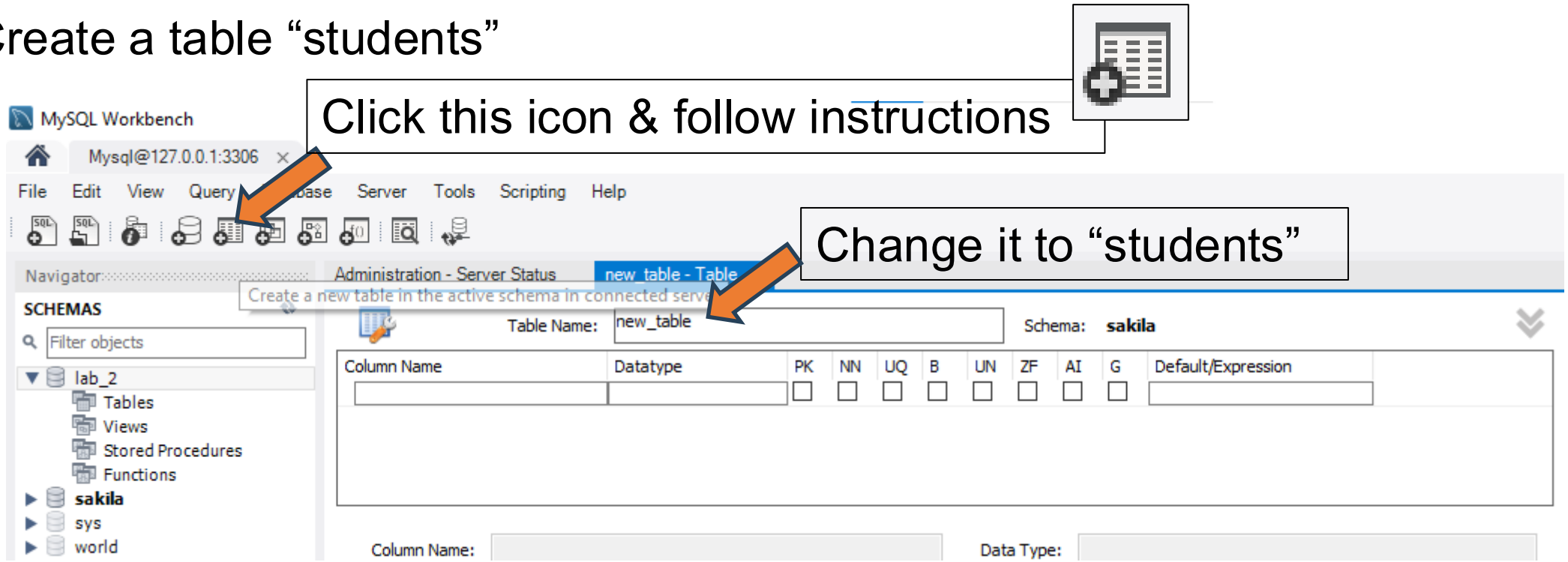


Click this icon & follow instructions



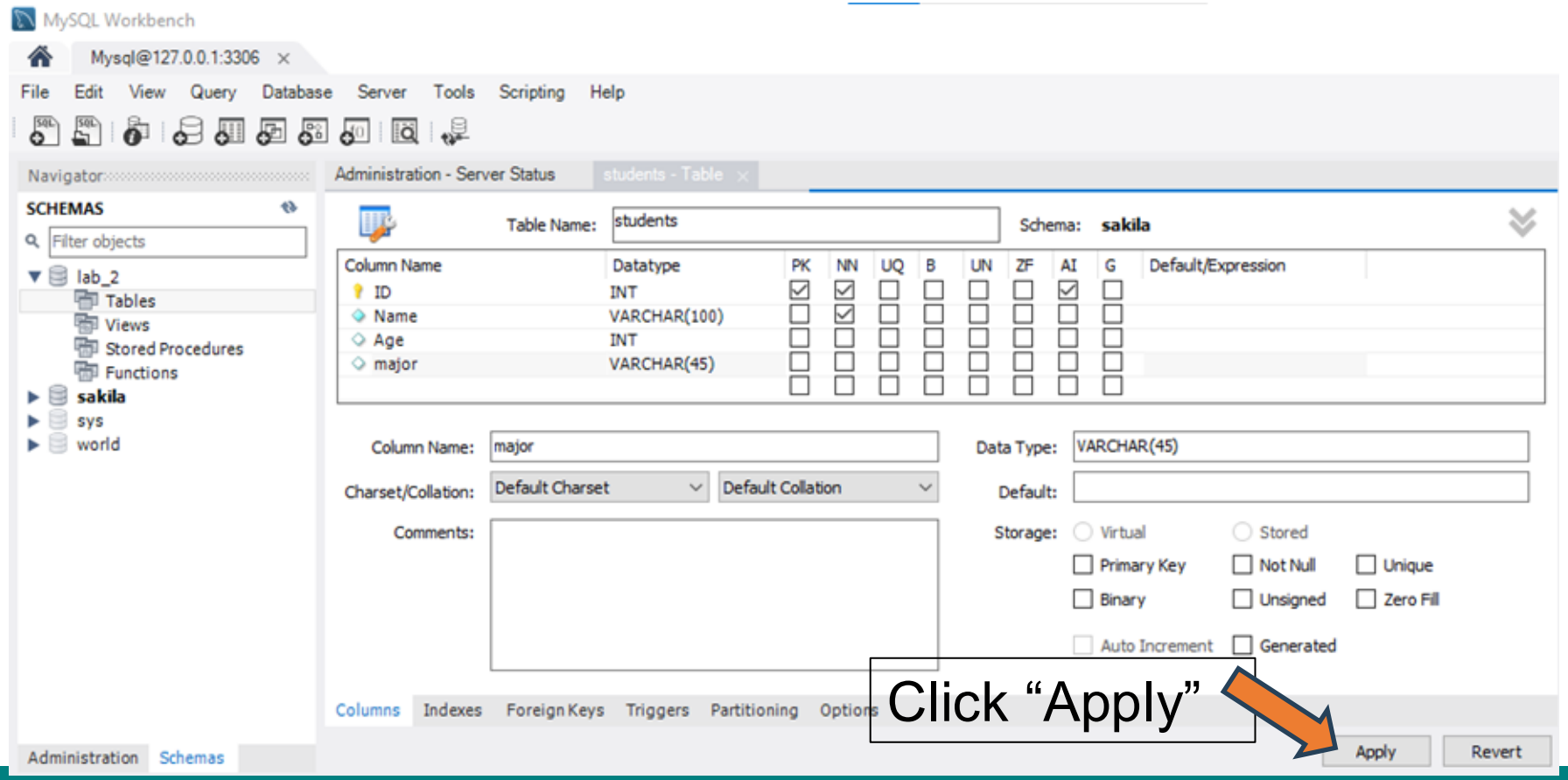
Connecting to MySQL Workbench and a tour of the interface (Contd.)

Create a table “students”



Connecting to MySQL Workbench and a tour of the interface (Contd.)

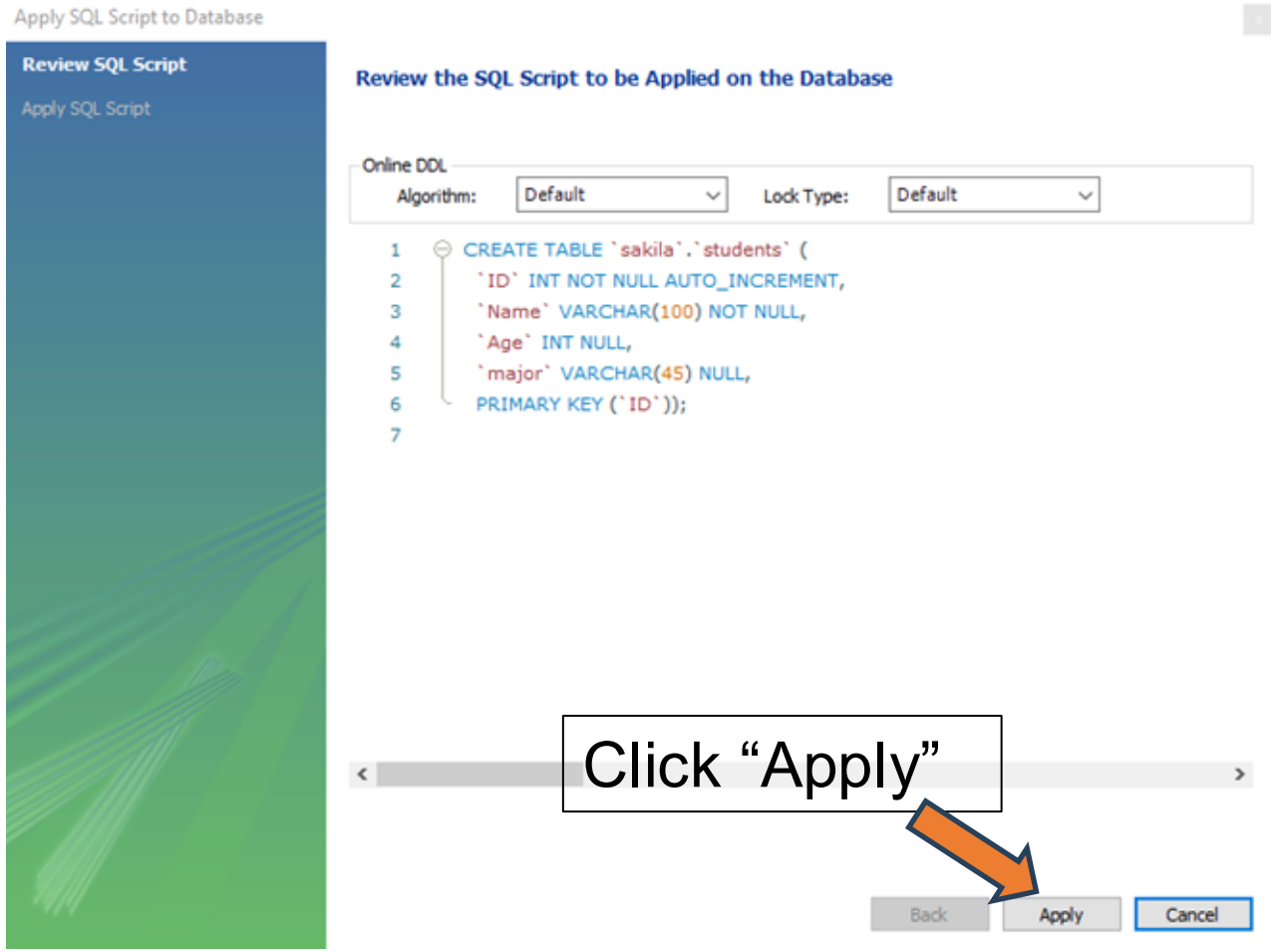
Create a table “students”



Connecting to MySQL Workbench and a tour of the interface (Contd.)

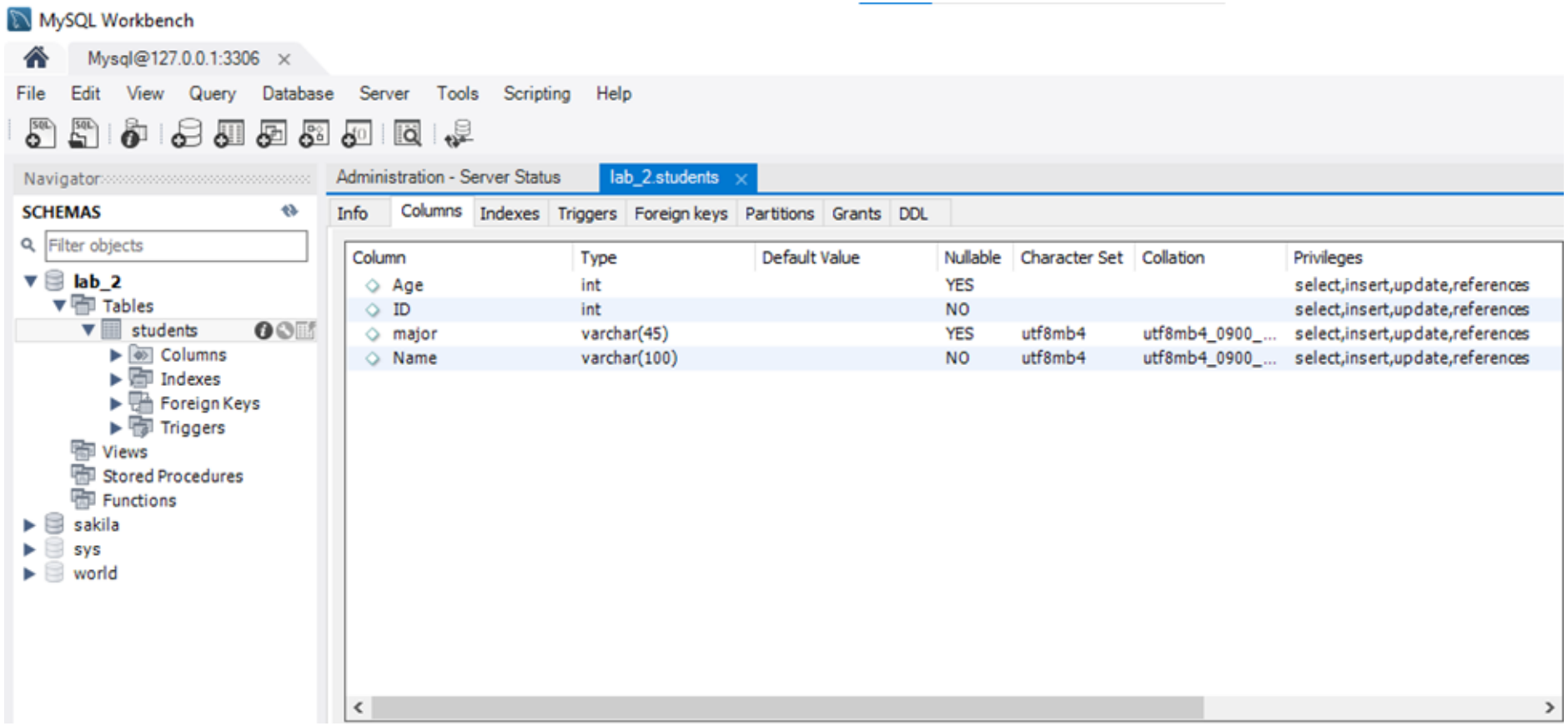
Create a table “students”

These Popup boxes are common in Workbench. Do not need to panic.



Connecting to MySQL Workbench and a tour of the interface (Contd.)

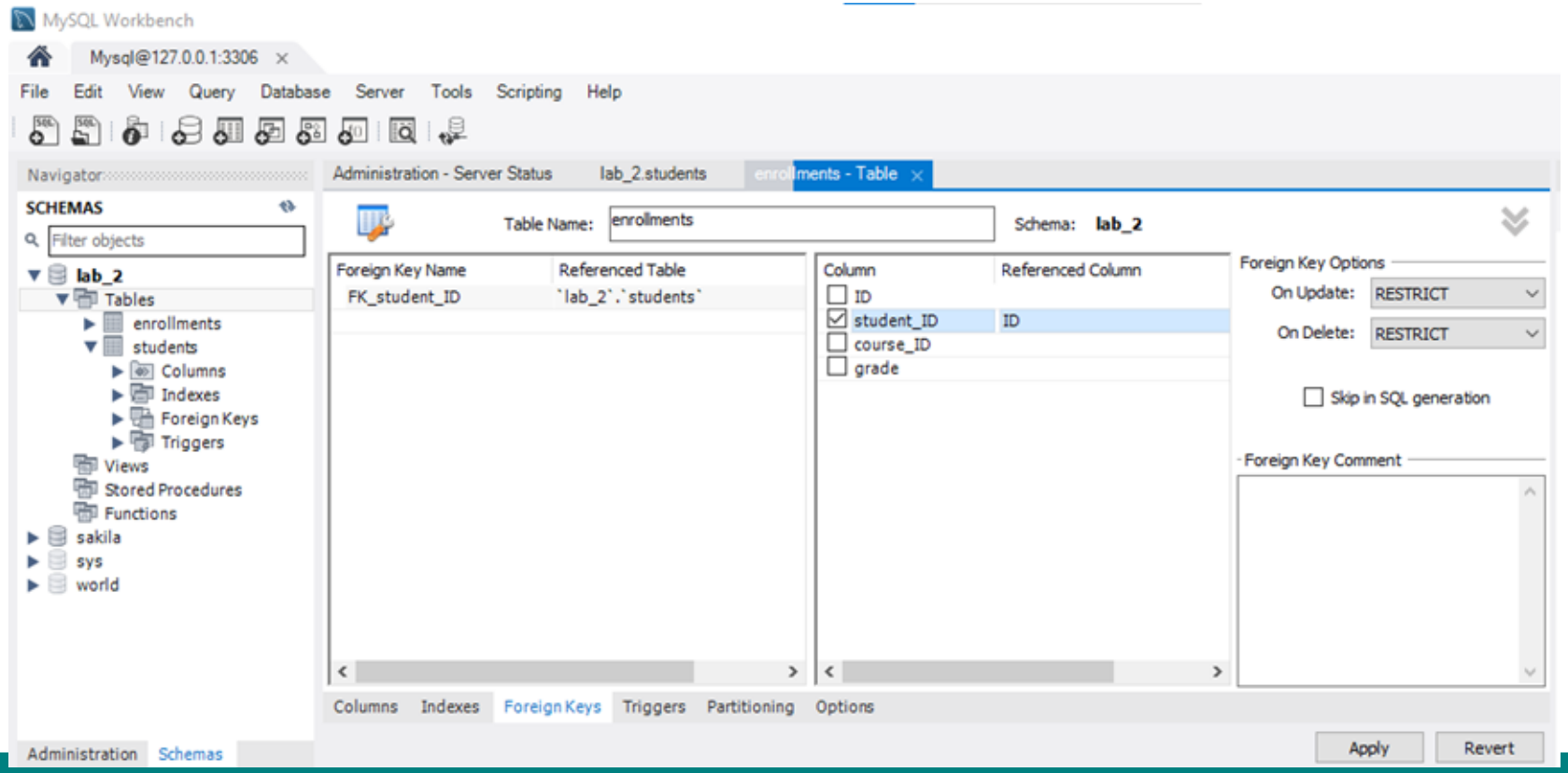
Create a table “students”



Connecting to MySQL Workbench and a tour of the interface (Contd.)

Create a table “enrollments” with a foreign key – Add Columns(ID, student_ID, course_ID and grade), then click tab “ForeignKeys”

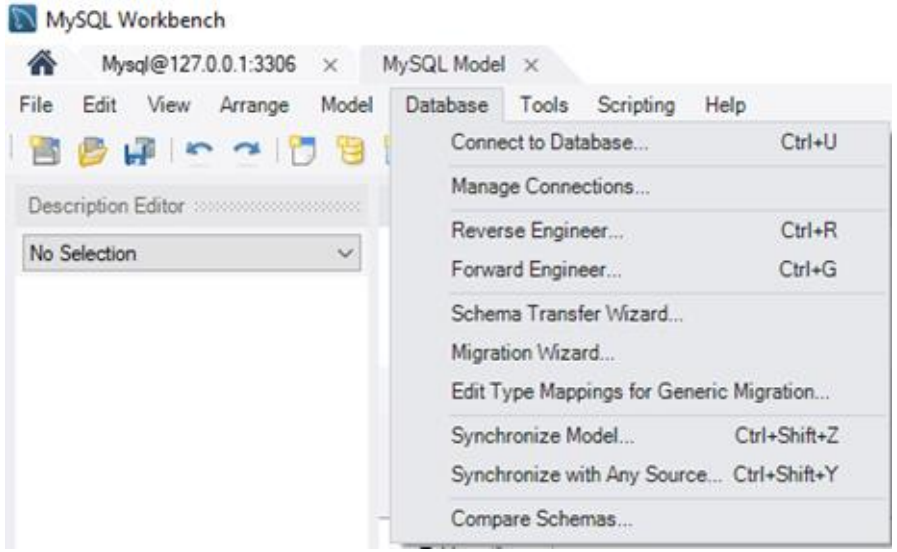
The FOREIGN KEY constraint stops you adding an enrollment for a student who does not exist.

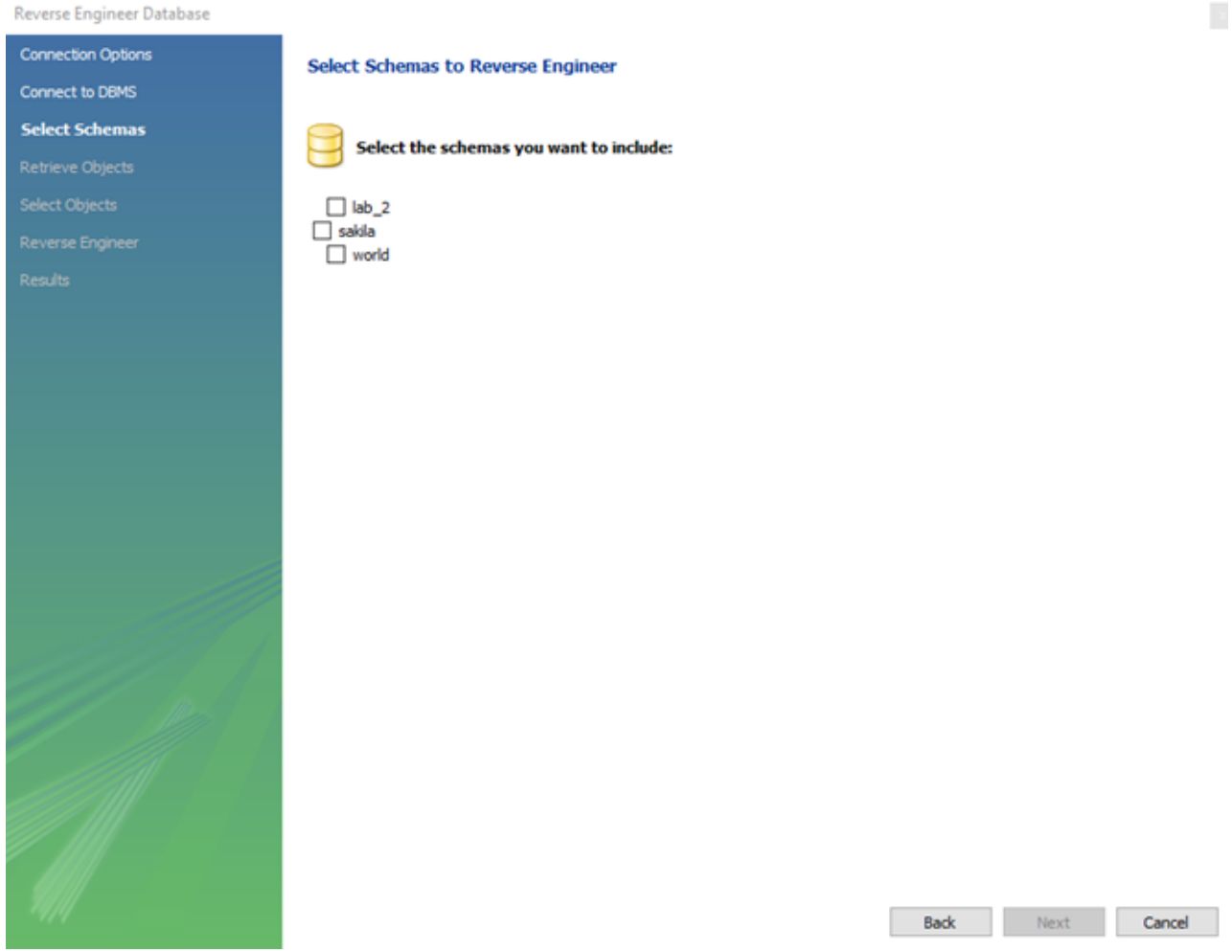


Connecting to MySQL Workbench and a tour of the interface (Contd.)

Create a MySQL Model

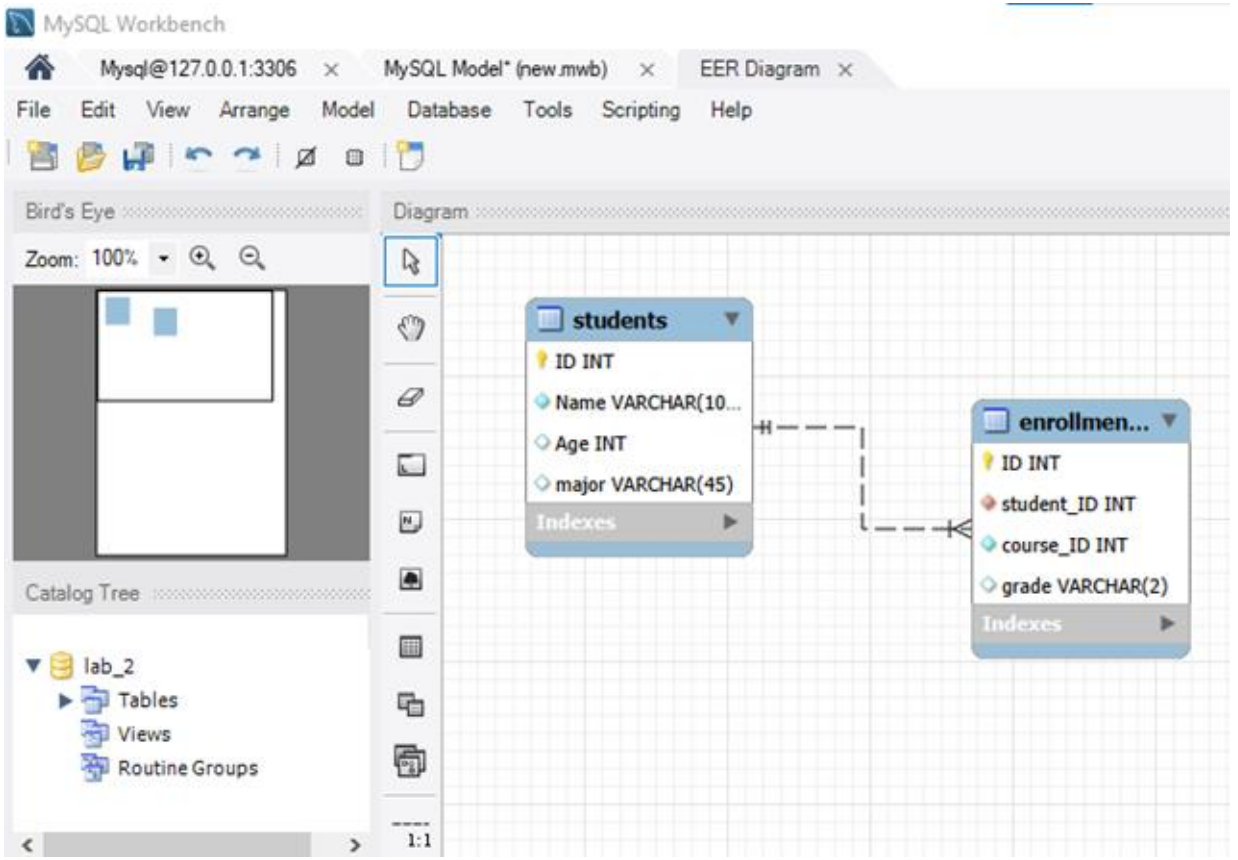
Go to home & select model, then from database ->Reverse Engineer. Select "lab_2"





Connecting to MySQL Workbench and a tour of the interface (Contd.)

Create a MySQL Model (EER)



Connecting to MySQL Workbench and a tour of the interface (Contd.)

Lab Exercise 4: Create a table (Using Workbench)

- Using your library design from Exercise 1, create the books table.
- Choose types, mark the PRIMARY KEY, and add NOT NULL where it makes sense.
- If you add a members or loans table, write a FOREIGN KEY for it.

Where We're Going Next

- You now have a schema: tables, types, keys and relationships.
- Lab 3: write SQL to fill these tables and query them — INSERT, SELECT, UPDATE, DELETE.
- Lab 4: connect this MySQL database to Node.js (mysql2), replacing Lab 1's in-memory array for good.
- **Good design today makes every query you write later simpler and faster.**

Summary

- Data modeling organizes data into tables of rows (records) and columns (fields).
- Primary keys identify rows; foreign keys link tables into relationships.
- ER diagrams picture the schema; normalization removes redundancy.
- Create databases, tables turns the design into SQL with types and constraints.
- **Complete Exercises 1–4 in class, then submit Lab Assignment #2.**

Thank you & Assignment time…